

Práctica 6: Esteganografía Segura Avanzada

Ocultamiento Permutado mediante Hashing SHA-256, Cifrado XOR y Dispersión Pseudoaleatoria (PRNG) sobre Contenedores BMP

Estudiante: Luis Angel Diaz Dominguez
Unidad Profesional Interdisciplinaria en Ingeniería y Tecnologías Avanzadas (UPIITA)
Procesamiento Digital de Imágenes / Multimedia

27 de septiembre de 2026

1 Introducción Teórica

En los métodos convencionales de esteganografía LSB (como los estudiados en la Práctica 5), los bits de datos se incrustan secuencialmente desde el primer píxel del contenedor. Esta linealidad los hace altamente vulnerables al análisis esteganalítico y a la extracción no autorizada.

La ****Esteganografía Segura**** desarrollada en esta Práctica 6 por **Luis Angel Diaz Dominguez** combina tres capas de protección criptográfica para garantizar confidencialidad e indetectabilidad:

1. **Cifrado Simétrico XOR:** Protege el contenido del mensaje antes de la incrustación.
2. **Hashing Criptográfico SHA-256:** Deriva una clave única y determinista a partir de una contraseña ingresada por el usuario.
3. **Dispersión Pseudoaleatoria (PRNG):** Utiliza la clave derivada como semilla (*seed*) para dispersar los bits del mensaje en posiciones no consecutivas de la imagen.

2 Arquitectura del Sistema de Seguridad

2.1 Derivación de Clave y Dispersión Pseudoaleatoria

El programa no selecciona las posiciones al azar de forma caótica, sino de forma determinista mediante la contraseña provista:

- **Hashing del Password:** La contraseña en texto claro se procesa con la función de resumen digestivo **SHA-256**:

$$H = \text{SHA256}(\text{Password}) \quad (1)$$

- **Generación de Semilla:** Se extrae un entero a partir del hash hexadecimal para alimentar el generador pseudoaleatorio:

$$\text{Semilla} = H_{\text{hex}} \pmod{10^8} \quad (2)$$

- **Sincronización Determinista:** Al usar la semilla única ($\text{PRNG}(\text{Semilla})$), el emisor y el receptor generan la misma secuencia exacta de índices de píxeles sin necesidad de transmitir la lista de posiciones. Un atacante que desconozca la clave no podrá determinar qué píxeles contienen información, incluso si conoce la arquitectura del algoritmo.

2.2 Incrustación de Cabecera de Longitud

Para permitir la extracción dinámica sin conocer previamente la longitud del texto, el sistema empaqueta primero un entero de 32 bits (4 bytes) que codifica el tamaño del mensaje cifrado (L). Los primeros 32 bits extraídos determinan cuántos bits adicionales deben recuperarse:

$$\text{Total Bits a Leer} = 32 + (8 \times L) \quad (3)$$

3 Código Fuente Completo en Python

A continuación se incluye la solución en Python desarrollada en el entorno Google Colab:

```
import struct
import os
import math
import hashlib
import random

# Practica 6: Esteganografia Segura
# Estudiante: Luis Angel Diaz Dominguez

# 1. FUNCIONES BASE
def leer_bmp(filepath):
    with open(filepath, 'rb') as f:
        data = f.read()
        offset = struct.unpack_from('<I', data, 10)[0]
        width = struct.unpack_from('<i', data, 18)[0]
        height = struct.unpack_from('<i', data, 22)[0]
        header = bytearray(data[:offset])
        pixels = bytearray(data[offset:])
        return header, pixels, width, height

def guardar_bmp(filepath, header, pixels):
    with open(filepath, 'wb') as f:
        f.write(header)
        f.write(pixels)

def calcular_psnr(original_path, stego_path):
    _, p_orig, _, _ = leer_bmp(original_path)
    _, p_steg, _, _ = leer_bmp(stego_path)
    mse = sum((a - b) ** 2 for a, b in zip(p_orig, p_steg)) / float(len(p_orig))
    if mse == 0:
        return float('inf')
    psnr = 10 * math.log10((255 ** 2) / mse)
    return psnr

# 2. HERRAMIENTAS DE SEGURIDAD XOR Y ALEATORIEDAD
def cifrar_descifrar_xor(datos, password):
    hash_base = hashlib.sha256(password.encode()).digest()
    clave_extendida = (hash_base * (len(datos) // len(hash_base) + 1))[:len(datos)]
    return bytes(a ^ b for a, b in zip(datos, clave_extendida))

def obtener_posiciones_aleatorias(total_pixeles, cantidad_bits, password):
    semilla = int(hashlib.sha256(password.encode()).hexdigest(), 16) % (10**8)
    random.seed(semilla)
    return random.sample(range(total_pixeles), cantidad_bits)

# 3. ALGORITMO DE ESTEGANOGRAFÍA SEGURA
def embed_secure(src_path, dst_path, mensaje, password):
    header, pixels, width, height = leer_bmp(src_path)
    msg_bytes = mensaje.encode('utf-8')
    msg_cifrado = cifrar_descifrar_xor(msg_bytes, password)

    # 4 bytes de longitud + mensaje cifrado
```

```

datos_a_esconder = struct.pack('>I', len(msg_bytes)) + msg_cifrado

bits = []
for byte in datos_a_esconder:
    for i in range(7, -1, -1):
        bits.append((byte >> i) & 1)

if len(bits) > len(pixels):
    raise ValueError("Imagen demasiado pequeña")

posiciones = obtener_posiciones_aleatorias(len(pixels), len(bits), password)
pixels_mod = bytearray(pixels)

for i, pos in enumerate(posiciones):
    pixels_mod[pos] = (pixels_mod[pos] & 0xFE) | bits[i]

guardar_bmp(dst_path, header, pixels_mod)

def extract_secure(stego_path, password):
    header, pixels, width, height = leer_bmp(stego_path)

    # Leer los primeros 32 bits (longitud)
    pos_longitud = obtener_posiciones_aleatorias(len(pixels), 32, password)
    len_bits = [pixels[pos] & 1 for pos in pos_longitud]

    msg_len = 0
    for b in len_bits:
        msg_len = (msg_len << 1) | b

    total_bits = 32 + (msg_len * 8)
    if total_bits > len(pixels):
        return "Error: Password incorrecto o imagen corrupta."

    todas_las_pos = obtener_posiciones_aleatorias(len(pixels), total_bits, password)

    try:
        pos_mensaje = todas_las_pos[32:]
        bits_mensaje = [pixels[pos] & 1 for pos in pos_mensaje]

        msg_cifrado = bytearray()
        for i in range(0, len(bits_mensaje), 8):
            byte = 0
            for bit in bits_mensaje[i:i+8]:
                byte = (byte << 1) | bit
            msg_cifrado.append(byte)

        return cifrar_descifrar_xor(msg_cifrado, password).decode('utf-8')
    except Exception:
        return "Error al descifrar el mensaje."

# 4. EJECUCIÓN
if __name__ == '__main__':
    if not os.path.exists('images'):
        os.makedirs('images')

    ORIGINAL = './images/volcan.bmp'
    STEGO_ADV = './images/stego_final.bmp'
    TEXTO = "Mensaje_Cifrado_UPIITA_2026"
    CLAVE = "mi_llave_maestra"

    try:
        embed_secure(ORIGINAL, STEGO_ADV, TEXTO, CLAVE)
        recuperado = extract_secure(STEGO_ADV, CLAVE)

        print("---RESULTADO PRÁCTICA 6---")
        print(f"Mensaje recuperado: {recuperado}")

```

```

print(f"PSNR: {calcular_psnr(ORIGINAL, STEGO_ADV):.2f} dB")
except Exception as e:
    print(f"Error: {e}")

```

4 Resultados y Análisis de Rendimiento

Cuadro 1: Parámetros y resultados del procedimiento de esteganografía segura

Parámetro de Control	Valor Registrado
Imagen de Origen	volcan.bmp (1920 × 1080, 24 bpp)
Imagen Resultante	stego_final.bmp
Clave Secreta (<i>Password</i>)	"mi llave maestra"
Mensaje Oculto	"Mensaje Cifrado UPIITA 2026"
Longitud de Datos Ocultos	4 bytes cabecera + 27 bytes texto = 248 bits
Fidelidad Visual (PSNR)	> 90.00 dB

- **Integridad de Datos:** El texto recuperado mediante la función `extract_secure` utilizando la contraseña correcta fue 100% idéntico al original.
- **Resistencia a Contraseñas Erróneas:** Al intentar extraer el mensaje con una contraseña diferente, la semilla generada por SHA-256 produce un conjunto de posiciones completamente distinto, resultando en datos aleatorios ininteligibles o en un fallo de validación de longitud.

5 Conclusión

El trabajo efectuado por **Luis Angel Diaz Dominguez** demuestra la superioridad de los esquemas esteganográficos guiados por claves criptográficas frente a los métodos estáticos LSB. Mediante la integración del algoritmo SHA-256 para derivación de semillas y la permutación pseudoaleatoria con `random.sample`, se elimina cualquier patrón espacial evidente en la imagen contenedora. Esta implementación alcanza un alto estándar en seguridad informática y procesamiento digital de imágenes al garantizar simultáneamente imperceptibilidad visual (PSNR > 90 dB) y robustez criptográfica.